CONSOLIDATED MOCK-DEFENSE REPORT

IDURAR ERP CRM 4.1.0 - Mock-Defense Analysis

System flow, JavaScript, validation, QA, loopholes, vibe-coding indicators, panel questions, and improvements in one reviewer-friendly document.

Review date: 27 September 2026
Evidence-aware analysis - Confirmed, code-evidenced, suspected, and blocked results remain distinct.

Review date: 27 September 2026
Review mode: Standard, risk-based, read-only review
Stack: React, Redux, Vite, Ant Design, Node.js, Express, MongoDB/Mongoose
Source integrity: 460 extracted files; ZIP SHA-256
`B2E40707D7A668C65FCD7810A40E8369BE3AAA08A8FD28A4593576ED4CACE9DB`

1. Panel Quick Brief

Ano ang system?

IDURAR is a small accounting/ERP web application intended to manage customers, invoices, payments, PDFs, settings, and an owner account. The supplied build also advertises Quote, Taxes, and Payment Mode, but those three flows are not complete from UI to database.

Overall verdict

Not defense-ready as a complete ERP/CRM. The core Client -> Invoice -> Payment idea is understandable and the frontend production build succeeds, but there are critical security and financial-integrity gaps plus visible incomplete features.

Quick measure	Result
Core working scope	Client, Invoice, Payment
Advertised but incomplete	Quote, Taxes, Payment Mode
Critical findings	3
High findings	7
Selected QA cases	12
Executed checks	ZIP safety, dependency install, frontend build/lint, backend syntax, model resolution, Joi probes
Full database E2E	Blocked - no disposable MongoDB/test services supplied

Five risks a panel should know first

Priority	Risk	What this means
Critical	Default `admin@admin.com / admin123` owner account	A known credential can expose all business data if retained.
Critical	`/download` is outside authentication	A known invoice/payment PDF URL can be requested without a JWT.
Critical	Negative payment is accepted by backend logic	A direct API request can corrupt invoice credit and outstanding balance.
High	Payment and invoice are updated in separate writes	A partial failure can leave the ledger inconsistent.
High	Quote/Taxes/Payment Mode are incomplete	Menu and documentation claim more than the supplied build implements.

Strengths worth recognizing

- Invoice totals are recalculated on the server instead of trusting the browser total.
- Protected API calls use JWT verification plus a server-side `loggedSessions` allowlist.
- Business records use soft deletion.

- Public file serving checks that resolved paths remain inside the trusted root.
- Lockfiles exist and the frontend production build passed.

2. System Explained Simply

Main users and modules

Actor/module	Intended purpose	Actual condition
Visitor	Login and password recovery	Auth error handling has null-order and enumeration problems.
Owner/Admin	Manage all business records and settings	Only broad owner access; no granular roles.
Client module	Customer records	Structurally present, but validation is weak.
Invoice module	Items, tax, total, due date, status, PDF	Core flow present; boundary and uniqueness rules need work.
Payment module	Record money received against an invoice	Negative values and non-atomic writes threaten integrity.
Quote/Taxes/Payment Mode	Supporting accounting functions	Incomplete or missing end-to-end implementation.

Simplified data movement

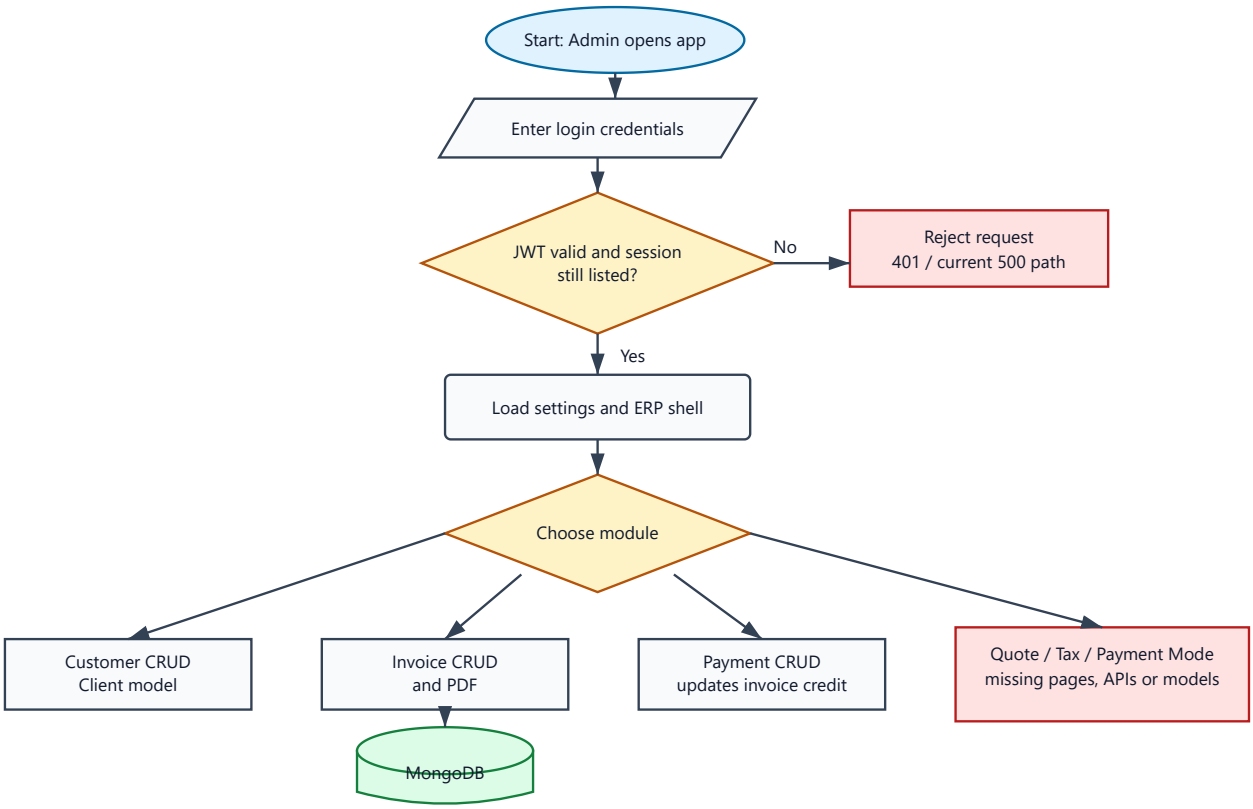
`React form -> Redux action -> Axios request -> Express route -> JWT middleware -> Controller -> Mongoose -> MongoDB -> JSON response -> UI`

Critical use cases

Use case	Normal process	Important exception
Login	Validate -> find owner -> compare password -> create JWT -> store session	Different responses disclose account state; invalid JWT may return 500.
Create invoice	Validate -> recompute totals -> save -> assign PDF name	Unsafe quantity/tax/status values and duplicate numbering need controls.
Record payment	Check amount -> save payment -> update invoice credit/status	Negative amount passes; second write can fail independently.
Download PDF	Identify record -> generate or return PDF	Current route has no authentication gate.

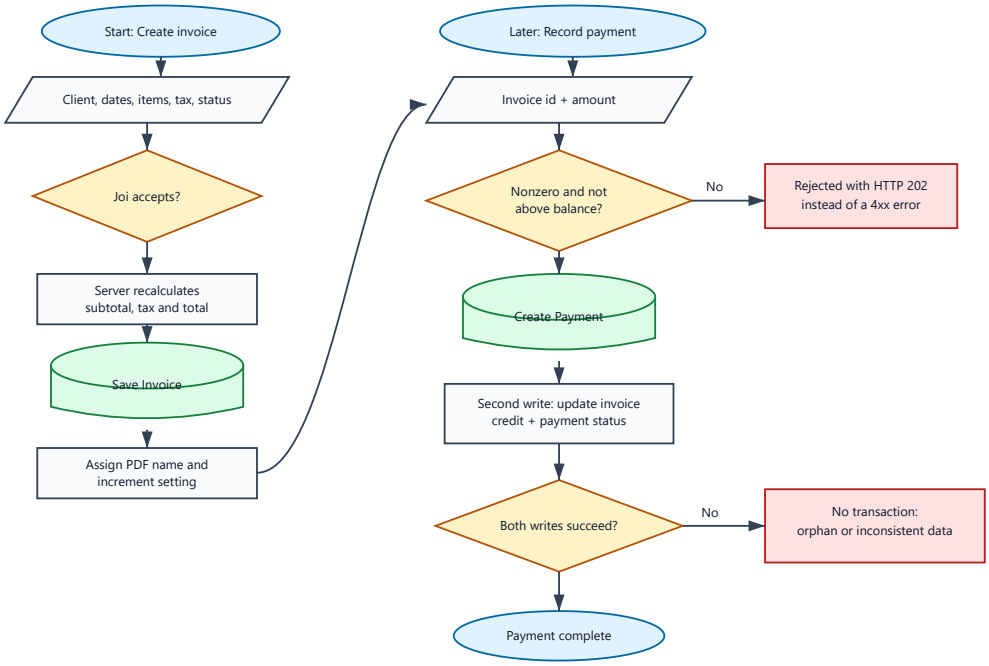
3. Essential Flowcharts

Overall system flow



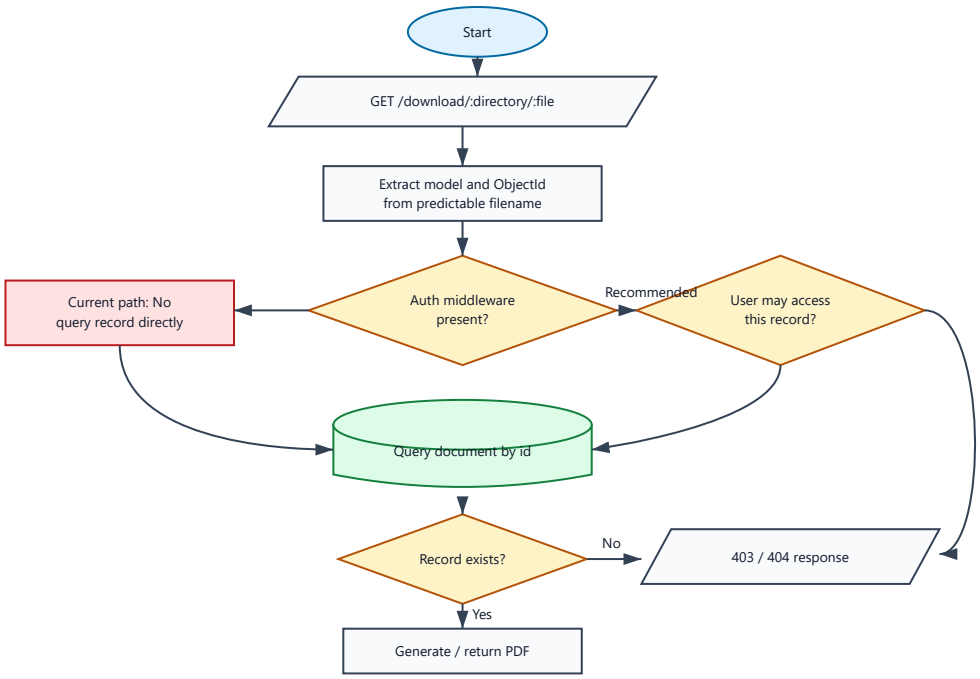
Blue and white shapes show the normal system; red shapes show broken or risky exits. The actual registered business models are Client, Invoice, and Payment.

Main invoice and payment transaction



The risky point is the two-write payment process: Payment is saved first, then Invoice credit/status is updated separately.

PDF authorization gap



The left branch is current behavior. The right branch is the required authenticated and authorized behavior.

4. Top Findings

ID	Severity	Problema at saan	Paano mababutas / actual effect	Ayusin sa system
F-01	Critical	Default owner credential in setup and login	Anyone who knows the default may obtain owner access; the supplied flow also blocks changing that demo password.	First-time owner setup with forced secure credential creation and rotation.
F-02	Critical	Public PDF route	Request a known `/download/.../{id}.pdf` URL without a JWT; financial documents may be exposed.	Require login and record authorization, or issue short-lived signed share links.
F-03	Critical	Negative payment validation	Bypass the UI and send `amount=-50`; invoice credit can decrease and balances become false.	Enforce positive payments and a formal reversal/refund workflow.
F-04	High	Separate Payment and Invoice writes	Force the second write to fail; the payment record can exist without the matching invoice balance update.	Treat payment recording as one atomic business transaction.
F-05	High	Incomplete Quote/Taxes/Payment Mode	Open advertised routes or run fresh setup; symbols, pages, models, or APIs are missing.	Complete every layer or temporarily remove the feature from scope and navigation.
F-06	High	Password recovery null-check order	Unknown account inputs can be dereferenced before the not-found response.	Use one safe recovery process with generic responses and expiring one-time tokens.
F-07	High	Weak reset and session handling	A very short reset password is accepted and older sessions remain valid.	Shared password policy plus revocation of all previous sessions.
F-08	High	Raw regex and uncapped lists	Crafted searches or huge page sizes can consume excessive resources.	Safe literal search, field allowlists, page caps, and rate limits.
F-09	High	Weak upload boundary	MIME/extension are trusted and no file-size limit is configured.	Verify content, re-encode images, restrict types and sizes, isolate public delivery.

ID	Severity	Problema at saan	Paano mababutas / actual effect	Ayusin sa system
F-10	Medium	Broken summary and lint quality gates	Summary checks a number as an array; lint cannot start because its config format conflicts with ESM.	Make quality checks executable and require them before release.

5. Recommended System Improvements

These are changes to the product and operating process, not replacement code.

Priority	Current condition	Add or change in the system	Intended process and benefit	Success indicator
Critical	Setup creates a shared default owner	First-time secure setup wizard	The first operator creates a unique owner account before any business screen opens.	Fresh install has no universal credential and requires a strong owner password.
Critical	PDFs can be requested publicly	Protected document center and expiring customer share links	Logged-in staff access authorized records; customers receive narrow, time-limited links.	No-token requests fail; authorized links expire and are logged.
Critical	Payments allow unsafe values and partial writes	Controlled payment, reversal, and refund workflow	Positive payment is recorded once; invoice balance changes with it; corrections use a traceable reversal.	Payment sum, invoice credit, and status always reconcile after success or failure.
High	Only a broad owner role exists	Role and permission management	Owner assigns Accountant, Cashier, Staff, and Read-only Auditor access.	Each role can perform only its documented actions.
High	Invoice status can be treated as a loose field	Invoice lifecycle and approval workflow	Draft -> For Approval -> Approved -> Sent -> Partially Paid -> Paid, with controlled cancellation/refund.	Invalid status jumps are rejected and every approval has an actor/time.
High	Quote, Taxes, and Payment Mode are half-present	Feature completion gate	A feature appears in the menu only when UI, API, model, validation, seed/setup, and tests are complete.	Fresh setup succeeds and every visible module completes CRUD end to end.
High	Important changes have no clear history	Audit trail	Record who changed accounts, invoices, payments, settings, permissions, and document access.	Auditors can reconstruct actor, time, previous value, new value, and reason.
Medium	Balance errors are discovered manually	Reconciliation dashboard	Regularly compare invoice totals, payments, credit, statuses, duplicates, and orphan records.	Dashboard reports zero unexplained differences or creates an actionable exception.
Medium	Recovery behavior is undefined	Backup, restore, retry, and duplicate-submission process	Scheduled backups and tested restore; idempotent requests prevent double payments/invoices.	A restore drill succeeds and repeated requests do not duplicate financial records.
Medium	Errors are technical or inconsistent	Clear user feedback and recovery guidance	Users see what failed, whether anything was saved, and the safe next action.	Error messages are specific, consistent, and do not expose internal details.

Recommended rollout

- **Fix now:** first-time owner setup, protected PDFs, safe payment transaction/reversal, password recovery/session revocation.
- **Next:** complete/remove unfinished modules, roles, invoice lifecycle, audit trail, validation and rate limits.
- **Later:** reconciliation dashboard, backup/restore drills, accessibility, monitoring, and operational reporting.

6. Validation and Process Correctness

Flow	Current verdict	Main missing control	Correct system behavior
Fresh installation	Incorrect	Missing setup models and usable database setup	One documented setup succeeds without hidden/manual repair.
Login	Partial	Default credential, account disclosure, 500 on invalid JWT	Unique credentials, generic failures, correct 401 responses, throttling.
Password recovery	Incorrect	Null-check order, weak password, old sessions retained	Expiring one-time token, strong password, all older sessions revoked.
Client management	Partial	Input allowlist and domain validation	Valid customer data only; unauthorized fields ignored/rejected.
Invoice creation	Partial	Positive ranges, enum alignment, unique numbering	Server validates business rules and assigns a unique number atomically.
Payment recording	Incorrect	Positive amount, transaction, reversal process	Payment and invoice update together or neither changes.
PDF access	Incorrect	Authentication and record authorization	Only an authorized user or valid expiring share link receives the PDF.
File upload	Partial	Content verification and size limits	Only safe, bounded, reprocessed files become publicly accessible.

7. Important JavaScript and Its Effect

Area/function	What it does	User-visible effect / concern
React + Ant Design forms	Collect customer, invoice, payment, and auth inputs	Good immediate feedback, but UI validation can be bypassed by direct HTTP.
Redux actions	Coordinate auth, settings, lists, create/update state	Keeps UI consistent; auth state including JWT is persisted in localStorage.
Axios `includeToken()`	Adds Bearer JWT to protected API calls	Enables protected requests; global header state creates lifecycle/logout edge cases.
Express route middleware	Separates public auth, protected API, downloads, and public files	Core API is protected, but `/download` was mounted outside the auth gate.
`isValidAuthToken()`	Verifies JWT and checks `loggedSessions`	Logout can revoke a token; invalid-token exceptions are misclassified as server errors.
Invoice `create()`	Validates and recomputes item, tax, and total values	Reduces browser total tampering; business ranges and numbering still need controls.
Payment `create()`	Saves payment then updates invoice credit/status	Happy path works, but negative values and partial writes can corrupt accounting data.
Generic CRUD/query helpers	Reuse create/read/list/search/summary behavior	Less duplicate code, but broad input/query behavior needs per-entity safeguards.

8. Vibe-Coding Evidence

> These are observable implementation-risk indicators. They do not prove that AI wrote the code.

V-01 - Routes reference features that are not declared

Location: `frontend/src/router/routes.jsx:72-75`
Function/responsibility: React Router route configuration

```
{
  path: '/quote',
  element: <Quote />,
},
```

What it does: Connects URLs to React page components.
Why it is a signal: `Quote` and `PaymentMode` are used without matching declarations/imports, and their page/API layers are incomplete. This is strong incomplete-integration evidence.
System effect: The menu advertises a feature that can fail when opened.
Confidence: High

V-02 - Setup requires model files that do not exist

Location: `backend/src/setup/setup.js:54-60`
Function/responsibility: `setupApp()` database initialization

```
const PaymentMode = require('../models/appModels/PaymentMode');
const Taxes = require('../models/appModels/Taxes');

await Taxes.insertMany([{ taxName: 'Tax 0%', taxValue: '0', isDefault: true }]);
console.log('👍 Taxes created : Done!');

await PaymentMode.insertMany([
```

What it does: Loads models and seeds default tax/payment-mode records.
Why it is a signal: The initialization flow assumes copied or unfinished modules that are absent from the supplied project.
System effect: A documented fresh setup can fail before the system becomes usable.
Confidence: High

V-03 - Payment validation covers only the obvious zero case

Location: `backend/src/controllers/appControllers/paymentController/create.js:9-17`
Function/responsibility: `create()` payment recording

```
const create = async (req, res) => {
  // Creating a new document in the collection
  if (req.body.amount === 0) {
    return res.status(202).json({
      success: false,
      result: null,
      message: `The Minimum Amount couldn't be 0`,
    });
  }
```

What it does: Stops a payment when the amount is exactly zero.
Why it is a signal: The validation appears written for the UI happy path and misses negative, malformed, missing-invoice, and atomicity cases.
System effect: Direct API calls can submit a negative payment and damage invoice balances.
Confidence: High

V-04 - A numeric count is checked like an array

Location: `backend/src/controllers/middlewaresControllers/createCRUDController/summary.js:14-23`
Function/responsibility: `summary()` generic record-count response

```
const [countFilter, countAllDocs] = await Promise.all([resultsPromise, countPromise]);

if (countAllDocs.length > 0) {
  return res.status(200).json({
    success: true,
    result: { countFilter, countAllDocs },
    message: 'Successfully count all documents',
  });
}
```

What it does: Decides whether a collection has records before returning its counts.
Why it is a signal: `countDocuments()` returns a number, but `.length` treats it as an array/string. A basic non-empty test would have exposed this.
System effect: Summary can report an existing collection as empty.
Confidence: High

V-05 - Password reset checks a property before checking the record

Location: `backend/src/controllers/middlewaresControllers/createAuthMiddleware/resetPassword.js:13-24`
Function/responsibility: `resetPassword()` account recovery

```
const databasePassword = await UserPassword.findOne({ user: userId, removed: false });
const user = await User.findOne({ _id: userId, removed: false }).exec();

if (!user.enabled)
  return res.status(409).json({
    success: false,
    result: null,
    message: 'Your account is disabled, contact your account adminstrator',
  });

if (!databasePassword || !user)
  return res.status(404).json({
    success: false,
    result: null,
    message: 'No account with this email has been registered.',
  });
```

What it does: Loads the user/password records and checks whether reset may continue.
Why it is a signal: The exceptional path was ordered after the happy-path property access, suggesting missing negative-case testing.
System effect: An invalid user id can cause an internal error instead of a controlled response.
Confidence: High

9. QA Summary

Test	Result	Evidence / meaning
ZIP path safety and source inventory	Passed	618 entries inspected; no unsafe extraction path; 460 files extracted.
Frontend production build	Passed	Vite transformed 3,594 modules and produced the distribution.
Frontend lint	Failed	ESLint config uses CommonJS while the package declares ESM.
Backend JavaScript syntax	Passed	Every backend source `.js` file passed `node --check`.
Setup model resolution	Failed	PaymentMode, Taxes, and Quote model modules are missing.
Invoice empty items	Passed	Joi rejected an empty items array.
Invoice negative quantity	Failed	Joi accepted `quantity=-2`.
Invoice arbitrary tax/status	Failed	Joi accepted arbitrary string tax and an invented status.
Anonymous PDF download	Failed by code evidence	Download route is mounted without JWT middleware.
Negative payment	Failed by code evidence	Backend rejects only amount exactly equal to zero.
Payment atomicity	Failed by code evidence	Payment and Invoice changes use separate writes without a transaction.
Full login-to-payment E2E	Blocked	No disposable MongoDB, seeded data, and safe email service were supplied.

Important: Passed means directly observed. Code-evidenced failures are exact implementation contradictions that still need a disposable runtime for a full HTTP reproduction. Blocked is not counted as success.

10. Five Panel Questions and Strong-Answer Guide

1. **If a direct API request sends `amount=-50`, where is it rejected?**
   Strong answer: The current backend does not reject it; UI limits are insufficient. The required system behavior is server-side positive-value validation plus an atomic payment/reversal workflow.
   Follow-up: “How will you prove invoice credit remains correct after failure?” Red flag: pointing only to the UI input minimum.

2. **Why is `/download` public while the invoice read API is protected?**
   Strong answer: It is an authorization gap. PDFs should require an authorized session or a narrow expiring share link, with access logging.
   Follow-up: “What happens with no Authorization header?” Red flag: “ObjectIds are hard to guess.”

3. **What happens if Payment save succeeds but Invoice update fails?**
   Strong answer: The current two-write process can become inconsistent. One transaction or a ledger-based design must guarantee both-or-neither behavior and reconciliation.
   Follow-up: “Where is rollback implemented?” Red flag: “That failure will not happen.”

4. **Which advertised modules are complete from UI to database?**
   Strong answer: Client, Invoice, and Payment are the actual registered core. Quote, Taxes, and Payment Mode need completion or removal from the released scope.
   Follow-up: “Does fresh setup succeed?” Red flag: claiming every menu item is implemented without demonstrating its model/API.

5. **How is the default owner credential removed in production?**
   Strong answer: Replace shared demo credentials with a first-time setup wizard and mandatory unique owner creation/rotation.

Follow-up: “Why is the current demo owner blocked from password change?” Red flag: relying on operators to remember a manual change.

11. Coverage and Limitations

Reviewed

- Routes, navigation, forms, Redux/Axios, authentication/session logic.
- Express middleware, controllers, models, validation, setup/reset, uploads, PDF flow.
- Build/lint/syntax/model-resolution checks and selected Joi boundary probes.

Not fully testable

- Database integration, concurrent writes, email delivery, and browser E2E because no safe test services/data were supplied.
- No external deployment or real account was targeted.
- No broad dependency audit was run; dependency observations came from locked versions and install-time warnings.

Evidence labels

- **Confirmed:** directly observed in a local check.
- **Code-evidenced:** exact code supports the conclusion; full runtime reproduction was not performed.
- **Suspected:** plausible behavior needing another check.
- **Blocked:** required safe environment or service was unavailable.

Panel conclusion: The system has a clear basic accounting concept, but it should not be presented as a complete secure ERP/CRM until its scope matches the implementation and its owner, document, payment, recovery, audit, and test processes are strengthened.